DevOps Lab: AWS Application Load Balancer (ALB) Path-Based Routing

Difficulty Level: Medium **Duration:** 120 Minutes

Scenario

Your company is deploying a new web platform comprised of three distinct microservices: - **Service A (Product Catalog):** Serves traffic under the URL path prefix `/app1`. - **Service B (Order Management):** Serves traffic under the URL path prefix `/app2`. - **Service C (User Profile):** Serves traffic under the URL path prefix `/app3`.

To ensure high availability and load distribution, each microservice must run on a separate EC2 instance deployed in a different Availability Zone (AZ) within the same AWS region. An Application Load Balancer (ALB) must be positioned in front of these services to serve as the single public entry point, routing requests dynamically to the correct target group based on the HTTP request URL path.

To secure this multi-tier architecture, direct internet access to the EC2 instances must be blocked. The instances must only accept HTTP traffic routed through the ALB.

Task Objectives

To set up this high-availability web architecture, you must perform the following tasks: 1. **Launch 3 EC2 Instances:** Provision three Ubuntu EC2 instances—each in a different Availability Zone (e.g., `a`, `b`, and `c` suffixes)—and name them according to the convention. 2. **Install Web Application Servers:** On each instance, configure a web service (e.g., Nginx, Apache, or Python HTTP server) serving content on port `80` under its corresponding path (`/app1`, `/app2`, or `/app3`). 3. **Provision Application Load Balancer:** Set up a public-facing Application Load Balancer (ALB) across multiple public subnets. 4. **Create Target Groups:** Configure three separate target groups (one for each microservice) with active HTTP health checks. 5. **Configure Path-Based Listener Rules:** Add a listener on port `80` to the ALB with routing rules that direct incoming traffic as follows: - Path `/app1*` routes to Target Group A - Path `/app2*` routes to Target Group B - Path `/app3*` routes to Target Group C 6. **Secure Security Groups:** Restrict the EC2 instances' security group rules to only allow inbound TCP port `80` traffic coming from the ALB's security group.

Requirements

1. EC2 Microservice Instances

- **Instance 1 (Service A):**
- **Name Tag:** `service-a-host-<your-labskraft-username>`
- **Availability Zone:** AZ1 (e.g., `us-east-1a`)
- **Service Path:** `http://<private-ip>/app1/` (Should return a simple message like "Product Catalog Service")
- **Instance 2 (Service B):**
- **Name Tag:** `service-b-host-<your-labskraft-username>`
- **Availability Zone:** AZ2 (e.g., `us-east-1b`)
- **Service Path:** `http://<private-ip>/app2/` (Should return a simple message like "Order Management Service")
- **Instance 3 (Service C):**
- **Name Tag:** `service-c-host-<your-labskraft-username>`
- **Availability Zone:** AZ3 (e.g., `us-east-1c`)
- **Service Path:** `http://<private-ip>/app3/` (Should return a simple message like "User Profile Service")

2. Application Load Balancer (ALB)

- **ALB Name:** `app-services-alb-<your-labskraft-username>`
- **Scheme:** Internet-facing
- **Listener:** Port `80` (HTTP)
- **Target Groups:**
- `target-group-app1-<your-labskraft-username>` (containing `service-a-host-<your-labskraft-username>` on port 80)
- `target-group-app2-<your-labskraft-username>` (containing `service-b-host-<your-labskraft-username>` on port 80)
- `target-group-app3-<your-labskraft-username>` (containing `service-c-host-<your-labskraft-username>` on port 80)

3. Security & Access Control

- **ALB Security Group:** Allows inbound HTTP (`80`) from Anywhere (`0.0.0.0/0`).
- **EC2 Instance Security Group:** Allows inbound HTTP (`80`) only from the **ALB Security Group ID**. It must reject direct HTTP traffic from any other source.

Expected Workflow

Sample Health Check & Routing Responses

When accessing the load balancer via its public DNS name: - Querying `http://<alb-dns-name>/app1/` should return: `text Product Catalog Service - Status: OK` - Querying `http://<alb-dns-name>/app2/` should return: `text Order Management Service - Status: OK` - Querying `http://<alb-dns-name>/app3/` should return: `text User Profile Service - Status: OK`

Deliverables

The final implementation must consist of:

Deliverable	Description
Three Running Instances	EC2 instances named `service-a-host-<your-labskraft-username>`, `service-b-host-<your-labskraft-username>`, and `service-c-host-<your-labskraft-username>` placed in three distinct Availability Zones.
Three Web Applications	HTTP servers serving the correct content under `/app1/`, `/app2/`, and `/app3/` on the respective hosts.
Application Load Balancer	Active ALB named `app-services-alb-<your-labskraft-username>` configured with target routing rules.
Three Target Groups	Target groups named `target-group-app1-<your-labskraft-username>`, `target-group-app2-<your-labskraft-username>`, and `target-group-app3-<your-labskraft-username>` configured on port 80 with passing health check metrics.
Secure Instance Security Group	Instance firewall rules that block direct internet access and permit traffic only from the ALB.

Technology Stack

Technology	Purpose
Amazon EC2	Microservice compute host nodes.
AWS ALB	Application Load Balancer orchestrating path-based traffic routing.
Security Groups	Stateful firewalls controlling ingress and egress traffic.
Ubuntu Linux	Guest operating systems.
Nginx / HTTP daemon	Web servers hosting the microservice endpoints.

Verification & Grading Criteria

Your infrastructure configuration will be automatically graded based on the following checks:

Test Case	Requirement	Validation Method	Marks
TC1	**EC2 Instances Provisioning**	Verifies three EC2 instances exist (`service-a-host-<your-labskraft-username>`, `service-b-host-<your-labskraft-username>`, `service-c-host-<your-labskraft-username>`), are running, and are spread across three distinct AZs.	4 Marks
TC2	**Application Load Balancer Setup**	Verifies `app-services-alb-<your-labskraft-username>` is active, has a public DNS name, and is listening on port 80.	4 Marks
TC3	**Target Groups & Path Routing**	Verifies three target groups (`target-group-app1-<your-labskraft-username>`, `target-group-app2-<your-labskraft-username>`, `target-group-app3-<your-labskraft-username>`) are configured with path-based listener rules forwarding `/app1*`, `/app2*`, and `/app3*` correctly.	4 Marks
TC4	**Security Group Restrictions**	Verifies instances block direct internet access and only allow inbound HTTP port 80 traffic from the ALB.	4 Marks
TC5	**End-to-End Routing & Health Status**	Verifies targets show as healthy and requesting the paths on the ALB DNS name returns successful service responses.	4 Marks

Total Score: 20 Marks

Optional Enhancements

For advanced practice, you can attempt to implement: - Configure HTTPS (SSL/TLS) traffic termination at the ALB. - Set up an Auto Scaling Group (ASG) behind the ALB for elastic scaling. - Configure customized static HTML error page responses (HTTP 502/504) on the ALB. - Deploy the microservices using Docker containers.

Real-World Use Case

This architecture forms the basis of modern containerized and VM-based microservices platforms. Deploying services across multiple availability zones prevents total platform outages from single datacenter disruptions. Path-based routing allows teams to use a single domain entry point to distribute traffic seamlessly to specialized backend service fleets, improving security, routing efficiency, and reducing public IP allocation costs.